

WEB PROGRAMMING FUNDAMENTALS

LAB 2 – CSS & JavaScript

Content:

- *Styling webpages with CSS:*
 - o *Using CSS selectors to select webpage elements.*
 - o *Using CSS properties to format the selected elements.*
- *Using JavaScript to program the web on the browser side:*
 - o *Using JS to process events and perform calculations.*
 - o *Combine JS, DOM, and CSS to style the webpage dynamically.*

Ex 1. Do the examples in the lecture.

❖ **Objective:**

- Learn how to use the CSS selectors.
- Use CSS properties to format the webpage elements (CSS reference: <http://bit.ly/1ftu5Sl>)
- Learn JavaScript programming language.
- Use JavaScript + DOM to change the page content and format.

❖ **Requirement:**

- Do the examples in the lecture, browse them, and observe the results.
- Make some changes to the examples and observe the results.

Ex 2. Using CSS box model.

❖ **Objective:** Using the CSS box model in designing webpages.

❖ **Requirement:** Create a webpage similar to [Figure 7](#) with the following properties:

- Outer box width: 400px
- Image: download at <http://bit.ly/1hscWan>.
- Image width: 340px
- Space between the image and the text below: 20px
- Text window height: 200px.

❖ **Hints:**

- Use the `<div>` tag to group the elements.
- The page contains 1 outer box (dark red border) with 1 heading, 1 image, and 1 inner box for displaying the text.

- Suggested steps:
 - 1) Create the boxes and insert the content into these boxes. Ignore the format in this step. Result of this step: <http://bit.ly/OGxF0k>
 - 2) Format the boxes:
 - Outer box: width 400px, background color *LightYellow*, border attribute *3px solid brown*. You can add paddings between the content and the border.
 - Image border: style *inset*, width *5px*, and *Orange* color.
 - Inner box (containing text): set the size, border, and background as described in the requirement. Note: use the *overflow* attribute appropriately to avoid the content flowing out of the box.
 - Inner box (containing text): set the size, border, and background color as described. Note: use the *overflow* attribute to set the text wrap and display the scrollbar.

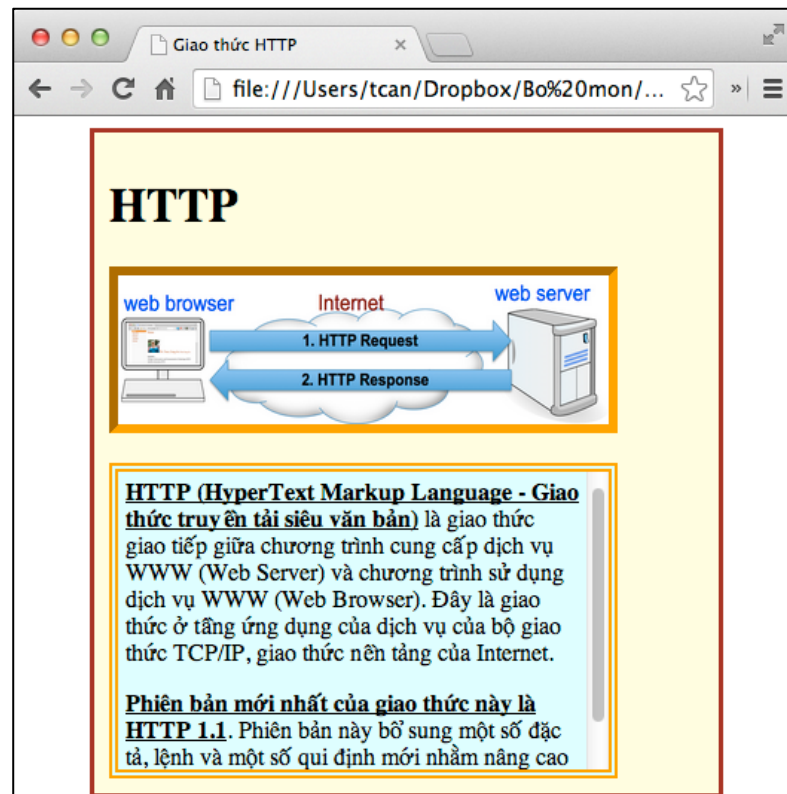


Figure 7. HTTP protocol introduction

Ex 3. Design a webpage to introduce your favorite songs.

❖ **Objective:** Create a webpage layout with CSS.

- Organize webpage layout into sections
- Set the space between elements using padding and margin.
- Use the *float*, *alignment*, and *clear* property.

❖ **Requirement:**

- Download file “my-music-plain.html” at <http://bit.ly/33ad4cu>, which is initially formatted with “my-music-basic.css” (download at <http://bit.ly/3cHM3l3>) and an empty CSS “my-music-layout.css” (download at: <http://bit.ly/2xtuV2l>).
- **Do the following tasks** to create the layout and style the webpage using CSS.
Note: “my-music-plain.html” had been linked to “my-music-layout.css”. So you just need to modify the “my-music-plain.html” and “my-music-layout.css” files.
 - 1) Organize the page content into sections and use CSS to format the page as shown in **Figure 8**:
 - Border width: 5px, *solid*.
 - Border colors: *green, red, orange, blue, and purple*.



Figure 8. My music – Step 1

- 2) Set the spaces and background for the page as shown in [Figure 9](#).
- The page background: use the following image <http://bit.ly/NCm2Xn>
 - Green box: white background; width: 80% of the browser width and center-aligned.
 - Blue box: background color #E8FBFB, padding 5px, space on the top 10px.

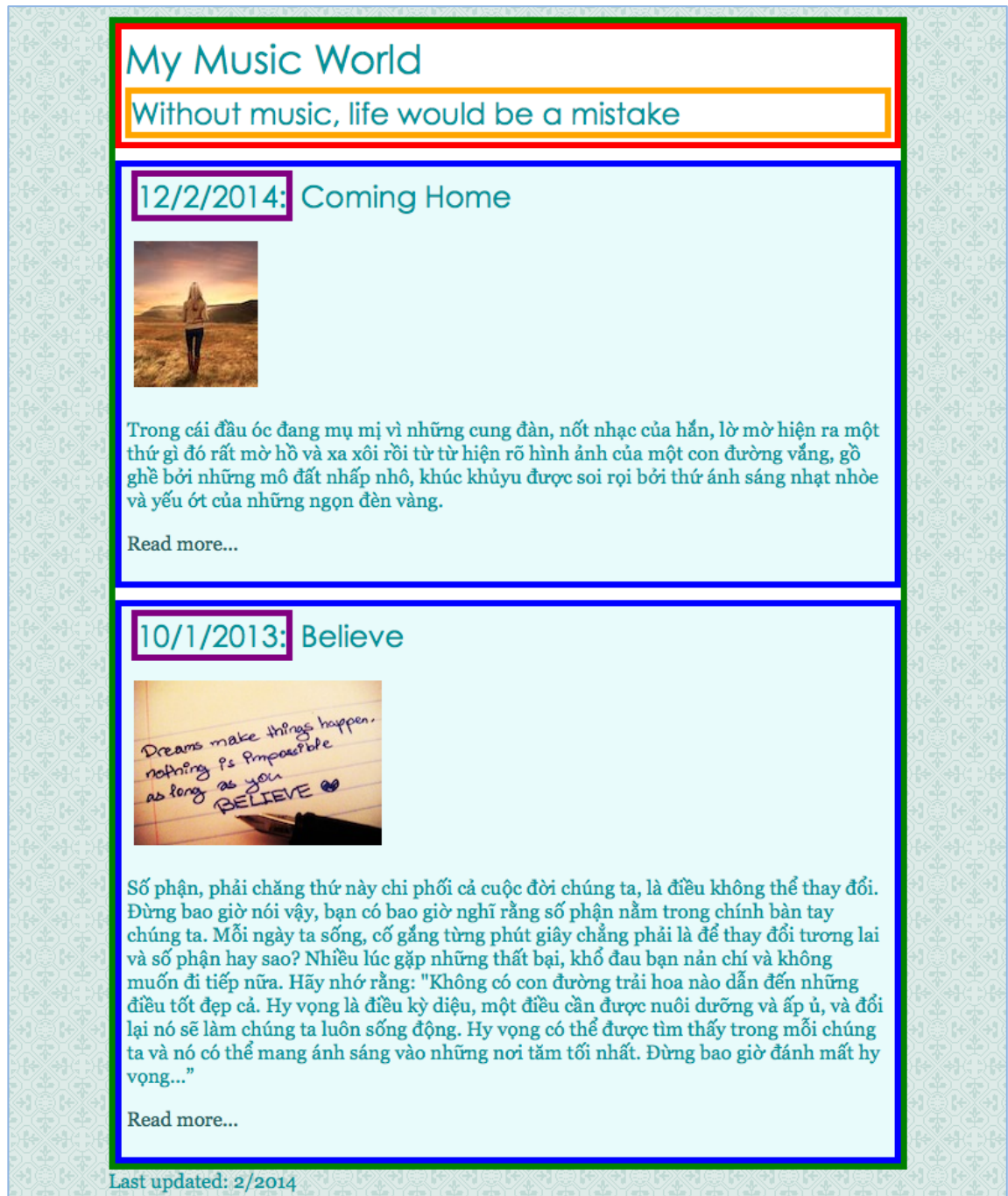


Figure 9. My music – Step 2

- 3) Align the element as shown in Figure 10.
- Right align the headings in the red box.
 - The images: float to the right (texts will wrap around the image).
 - Texts: justify alignment.



Figure 10. My music – Step 3

- 4) Change the border and background of the boxes, as shown in [Figure 11](#).
- Green box: change to the *white* border, *10px* thick.
 - Blue box: change the border color to *#C2E9E9*, *4px* thick and round the corners *15px* (*border-radius: 15px*).
 - Purple box: change the border color to *cyan*, *dashed* style, *2px* thick, and only show the *bottom* border.
 - Red box: change the background to *#A8F0F0* and remove the border.
 - Heading in the orange box: reduce the size to *14px*.



Figure 11. My music – Step 4

5) Add a list of **useful links** and set the background image for the content box, as shown in [Figure 12](#).

- Use the following HTML code to create the Useful Links list:

```
<h2>Useful Links</h2>
<ul>
  <li><a href="">...</a></li>
  <li><a href="">...</a></li>
  <li><a href="">...</a></li>
</ul>
```

- Add a border and background for the Useful Links and make it float to the right, as shown in [Figure 12](#).

- Background image for the content box: <http://bit.ly/1mMNNsR>



Figure 12. My music - Final page

❖ **Hint:**

1) Two basic steps for addressing the exercise requirements:

- a. Place the elements that need to be formatted into containers (using `<div>` or `<span>`) and set the container *class* or *id* attribute so that we can select and format them. For example:
 - Place all elements, except the last paragraph (last updated...), into a `<div>` and set its *id* to `"container"`.
 - Place two first headings (in the red box) into a `<div>`, and the second heading (in the orange box) is placed in another inner `<div>`. Set the *id* for these divs (e.g., *header* and *subheader* correspondingly).
 - Each song's introduction is placed into a `<div>` and sets its class to `"allentries"`.
 - As each song is separated from others and formatted similarly, placed each song into an inner `<div>` and set these divs to the same class, e.g., `"entry"`.
 - Do similarly for the dates. However, as they are displayed inline, put them into spans instead and assign them to the same class `"date"`.
- b. Write CSS rules in the file `"my-music-layout.css"` to create the border for elements as required. For example, to create the border for the container and date elements, we use the following rules:

```
#container {  
    border: 5px green solid;  
}  
.date {  
    border: 5px purple solid;  
}
```

2) Some properties used in this exercise:

- Background color: the *background-color* property.
- Background image: the *background-image* property
- Center the page and create the left and right margins (10% for each): the *margin-left* and *margin-right* properties of the body.

Write your own selectors to select the appropriate elements.

3) Alignment and floating:

- Alignment: use the *text-align* property of the *#header* and *.entry*
- The image inside each entry: set the *float* property of the images to the right.

- 4) Change the border and background:
 - Rounded border: use the *border-radius* property.
 - Dash line under the dates: replace *border* property in the CSS rule for the dates by *border-bottom-style* and *border-bottom-width*.
 - Other requirements: do by yourself ☺.

- 5) Add Useful Links and set the background image:
 - Add HTML code to create Useful Links and put them into a `<div>` tag. Assign an *id* to the div tag, e.g., “*navigation*”. This div must be placed outside the div *#allentries* and *#header*.
 - Create CSS rules to format the div *#navigation*:
 - Border, background, margin, and padding: similar to the previous steps.
 - Set the *width* of the div to 150px.
 - Set the *float* property of the div to make it float to the right.
 - When the div *#navigation* float to the right, the divs containing entries will float around and overlap the div *#navigation*. Therefore, to avoid the overlap as in the requirement, increase the left margin of the div *#allentries* (it should be greater than the width of the div *#navigation*, about 180px).
 - You can modify the content of the div *#navigation* to make the webpage more interesting.

Ex 4. Write a webpage to solve the quadratic function using JS.

❖ **Objective:**

- 1) Write basic JS code
- 2) Access form elements using JS and DOM

❖ **Requirements:**

- 1) Design a webpage as shown in [Figure 13.1](#)
- 2) When the user clicks on the button “Giải phương trình”, a JS code will be called to calculate the delta and solution x_1 and x_2 .
- 3) The computed values will be displayed on the page as in [Figure 13.2](#)
- 4) **Advanced:** Check the form elements’ value before solving the equation and show the appropriate messages if the input values are invalid (see [Figure 13.3](#)).

❖ **Hints:**

- 1) The webpage has 1 form, which contains 3 textfields and 1 button. To show delta, x_1 , and x_2 values, we insert empty `<span>` tags. We must set the *id* property for these spans so that we can access these spans later using this property. Between the span to display delta and x_1 , we insert a `<p>` tag to

display the message about the number of solutions. This `<p>` tag is also assigned an id.

- 2) Write a JS function to solve the quadratic function and call this function in the *onclick* event of the “Giải phương trình” button. The JS function accesses the form elements to get the value of *a*, *b*, and *c* variables using the *document.getElementById()* function. It also uses DOM to set the delta value, message on the number of solutions, and *x1* and *x2* values.
- 3) Write code to check and validate the input values. To display the message when the input value is invalid, insert a `<p>` tag next to each textfield. You can also make a further improvement so that the element containing invalid input will be focused (by calling the *focus()* method of the element).

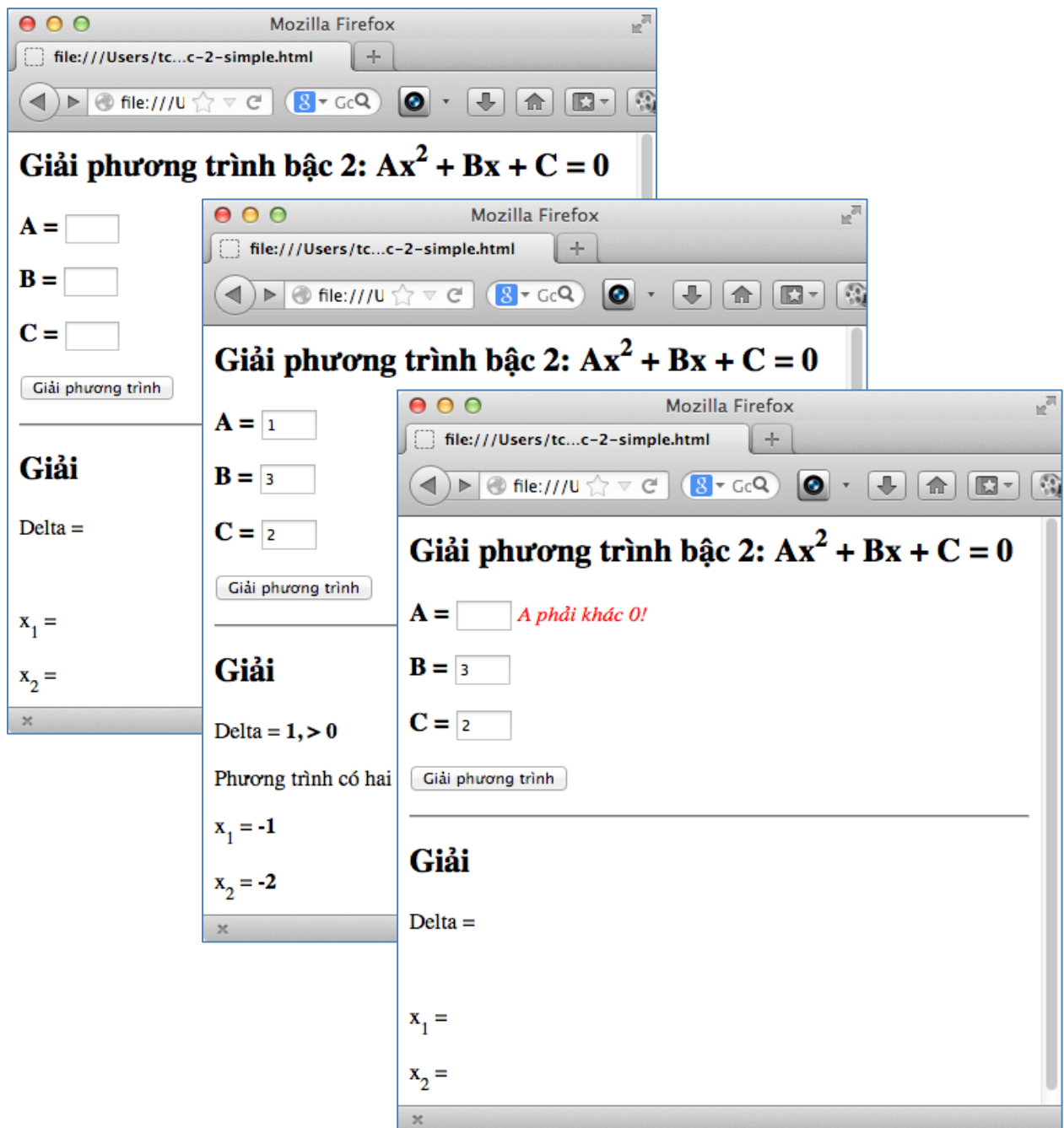


Figure 13. Quadratic function solving application

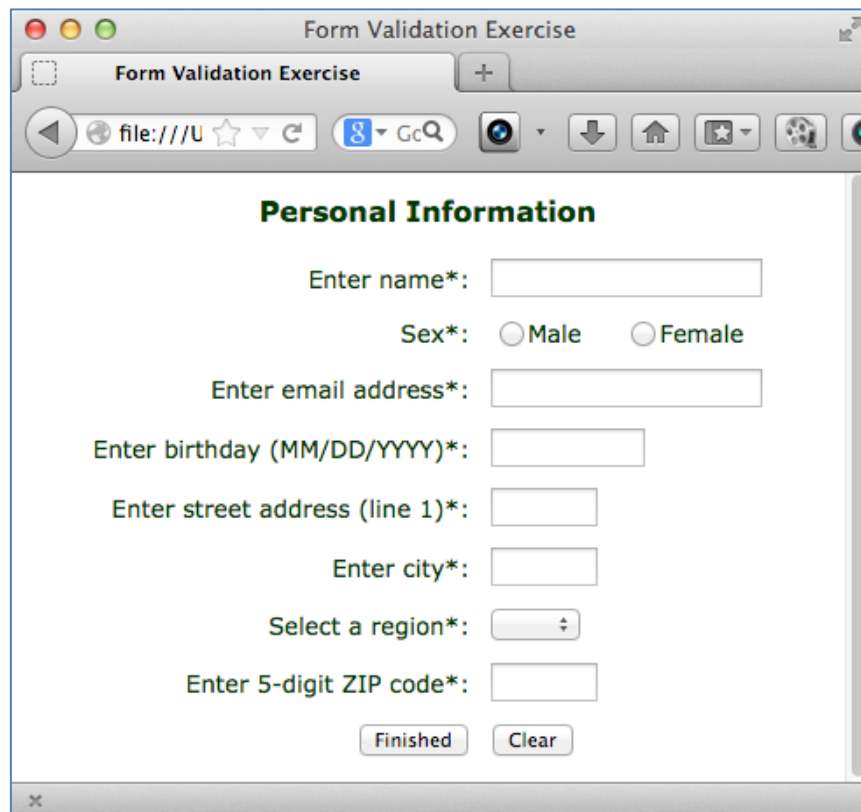
Ex 5. Create a form for user registration with validation.

❖ **Objective:**

- 1) Use JS to validate the data in the browser using regular expression.
- 2) Show the error message using DOM.

❖ **Requirement:**

- 1) Create a form as shown in [Figure 14](#).
- 2) When the Clear button is clicked: clear the content of all elements.
- 3) When the Finish button is clicked: validate the form element as follows:
 - All mandatory fields must be non-empty.
 - Email: must contain one "@", before "@" is an account name, after "@" is a domain name, the domain name has at least one "."
 - DOB: should be in mm/dd/yyyy format.
 - Zip code: includes exactly 5 digits.



The screenshot shows a web browser window with the title "Form Validation Exercise". The browser's address bar shows "file:///U". The page content is a form titled "Personal Information". The form contains the following elements:

- A text input field labeled "Enter name*:".
- A "Sex*" label with two radio buttons: "Male" and "Female".
- A text input field labeled "Enter email address*:".
- A text input field labeled "Enter birthday (MM/DD/YYYY)*:".
- A text input field labeled "Enter street address (line 1)*:".
- A text input field labeled "Enter city*:".
- A dropdown menu labeled "Select a region*:".
- A text input field labeled "Enter 5-digit ZIP code*:".
- Two buttons at the bottom: "Finished" and "Clear".

Figure 14. User registration form

Extra exercises

Ex 6. Modify [Ex 5](#) in Lab 1, and use CSS to format the webpage.

❖ **Objective:** use CSS to format the webpage and compare between 2 methods (using HTML vs. CSS).

❖ **Requirement:**

- 1) Open the VC page in Lab 1 and remove all formats (except the table size).
- 2) Use CSS to format the webpage to make it similar to [Figure 15](#).

❖ **Hint:**

- 1) Text color and font: select the page body and use the *color* property to set the text color (#454545) and *font-family* property to set the font face (Georgia).
- 2) The top header: set the *font-variant* property value to small caps. Other formats for this header: observe and do it by yourself.
- 3) The 2nd level header: set all of them to the same class and the following properties for this class: text-align (left), color (black), padding-top, padding-bottom, font-size, font-weight, and border-top.
- 4) The right column: use the *text-align* property to specify the horizontal alignment and the *vertical-align* property to specify the vertical alignment.
- 5) Space between 2 columns: in Lab 1, we used the *cellpadding* property to specify the space between 2 columns. However, this property affects all 4 sides of a cell. So, increasing the space between two columns also increases the space between rows. CSS allows us to adjust the space on each side of a cell separately, and thus we will use it to increase the space between 2 columns while still keeping the space between the rows unchanged. To increase the space between the table columns, we increase the right padding for the cells in the 1st column. The key problem is to create a selector that can select all the cells of the 1st column so that we can set their *padding-right* property. Alternatively, we can assign these cells to a class and select this class to apply the format. The first solution is better than the second one.

CURRICULUM VITAE



Personal Information

Name	TRAN CONG AN
Address	College of Information and Communication Technology CanTho University
Telephone	(+84) 7103 831301
Email	tcan@cit.ctu.edu.vn
Nationality	Vietnamese
Date of birth	23/12/1978

Education Qualification

2013	PhD in Computer Science , Massey University, New Zealand Thesis title: <i>"Symmetric Parallel Class Expression Learning"</i>
2007	MSc in Computer Science , Asian Institute of Technology, Thailand Thesis title: <i>"Ontology and Reasoning for Decision Making in the Portfolio Approach to e-Procurement"</i>
2001	MSc in Computer Science , CanTho University, Vietnam Thesis title: <i>"Disk-quota management for WinNT OS"</i>

Employment

9/2001 - Now	Lecturer College of Information and Communication Technology, CanTho University, Vietnam
7/2013 - 12/2013	Research Assistant Massey University, NewZealand
8/2007 - 11/2007	Research Assistant ArguGRID Project, Asian Institute of Technology, Thailand
9/2001 - 12/2002	Programmer/Project Manager CanTho University Software Center (CUSC), CanTho University, Vietnam
9/2001 - 5/2005	Invited Lecturer CanTho University Software Center (CUSC), CanTho University, Vietnam

Grants & Awards

2012	ECCAI Travel Grant to the ECAI12 The European Coordinating Committee for Artificial Intelligence (ECCAI)
2008-2012	PhD Scholarship Ministry of Education and Training of Vietnam
2008-2013	PhD Research Grant Massey University, NewZealand
2007	The Hisamatsu Prize for Outstanding Academic Performance in Field of Computer Science (batch 2005-2007) Asian Institute of Technology, Thailand
2005-2007	MSc Scholarship Ministry of Education and Training of Vietnam
1998-2001	4 x Outstanding Undergraduate Student Award CanTho University, Vietnam

Last updated: 2/2014

Figure 15. Curriculum Vitae - CSS

Ex 7. Design a film review webpage.

❖ **Objective:**

- 1) Use the multi-column layout in CSS3.
- 2) Use some advanced features of CSS to format a webpage.

❖ **Requirement:** download an unformatted webpage and the images used in the webpage at <http://bit.ly/1kjjqMj>. Make the following modifications on files “film-review.html” and “film-review.css” so that the webpage looks similar to **Figure 16**:

1) General setting:

- Background image: “images/background.png”.
- Font: *Verdana, Tahoma* và size là *8pt*.
- Page margin: 0px and no space between the page content and the margin (see Figure 16).

2) **Banner:**

- Banner image: “images/rbg.png”.
- Alignment: center.

3) The **heading** below the banner:

- Font face: *Tahoma, Verdana*; size: *24pt*; alignment: *center*.
- Shadow: *3px* on the bottom, *3px* on the right, and shadow color is *#999999*.

4) Put the content below the heading in a box 800px wide and set the following properties for the box:

- Alignment: center.
- Border: 4px width, gray color, solid style, corners rounded 20px.

5) Format of the **General Overview** section (on the right):

- Width: 250px, background color: *#A2B964*.
- No space between the image and the box margin.
- Space between text and margins: 10pt.
- Font: *Arial*, size: 8pt.
- Film information (starring, director,...): created using *<dl>*, *<dt>*, *<dd>* tags with the following properties:
 - Heading (*<dt>*): font style bold and space to the previous element *10pt*.
 - Information (** and **): no bullet and no indentation.

Abraham Lincoln: Vampire Hunter (2012)

32%

"Expect no intelligence or subtlety here, but if you're willing to put up with the sheer ridiculousness of it all, you might enjoy wallowing in Bekmambetov's shameless exhibition of narrative lunacy and technical fireworks in this movie."

Frank Swietek
One Guy's Opinion

"This movie just made Steven Spielberg's life easier because his Lincoln biopic coming out in December will definitely be the better of these two films. This could have been fun, it never settles in what it wants to be and ultimately fails."

Kristian Harloff
Schmoes Know

"The result is a film that feels more like the ridiculous Will Smith Wild Wild West and less like Bekmambetov's clever Night Watch."

Beth Accomando
KBPS.org

"The feature doesn't have the twinkle in its eye to accurately blend SuperLincoln with TrueLincoln into a loving, ax-swinging whole."

Brian Orndorf
BrianOrndorf.com

"It stinks!"

Jay Sherman (unemployed)

"If ever there was an 'either you're in or out' movie in 2012, it's definitely this one and if you're in, it just might be the most guilty pleasure of the year."

Jeffrey Lyles
Lyles' Movie Files

"Starting well and culminating in a lackluster second half, it is far too streamlined and spotty to satisfy as anything more than a pulpy slice of summer movie inconsequence."

Dustin Putman
DustinPutman.com

"YOUR REVIEW HERE"

NAME
PUBLICATION

"YOUR REVIEW HERE"

NAME
PUBLICATION

GENERAL OVERVIEW

STARRING
 Benjamin Walker
 Anthony Mackie
 Rufus Sewell
 Dominic Cooper

DIRECTOR
 Timur Bekmambetov

RATING
 R

THEATRICAL RELEASE
 June 22, 2012

MOVIE SYNOPSIS
 Abraham Lincoln: Vampire Hunter brings to the screen the secret life of our nation's favorite president... as history's greatest hunter of the undead.

MPAA RATING
 R, for violence throughout and brief sexuality

RELEASE COMPANY
 20th Century Fox

RUNTIME
 105 mins

GENRE
 Action & Adventure, Horror, Mystery & Suspense, Science Fiction & Fantasy

BOX OFFICE
 \$11,245,853

LINKS
[RT Reviews](#)
[IMDB Page](#)
[RT Home](#)
[CSE 190 M](#)

(1-10) of 88

Figure 16. Film review page

6) Ratings and reviews:

- Width: 550px; divided into 2 sections: the **top section** displays the dislike rate, and the **bottom section** shows user reviews.
- Top section** format:

- Height: 83px; background image: “*images/rbg.png*”.
 - Text font size: 48pt, bolded and red color.
 - **Bottom section** format:
 - 2-column layout; column width: 47% of the rating and review box.
(multiple column div tutorial: <http://bit.ly/1i1q3Td>).
 - Space between review boxes and their container: 10pt
 - Review text:
 - Font size: 8pt, bold.
 - Background color: “#E8DC9B”
 - Border: 2px width, gray color, solid line, rounded corner 10px.
 - The image inside the review box: floats on the left, space from text 5px.
 - Reviewer information:
 - Avatar: floats on the left of the text, space from text 5px.
 - The 2nd line (publication): italic.
 - Space from the reviewer information to the next review text: 20pt.
- 7) Footer: displays the total number of reviews and the current review number:
- Background color: #A2B964.
 - Space from text to the border: 5px.

Note: You can only add properties such as *id*, *class*, etc., for the element in the file “*film-review.html*”. Other modifications are not allowed. All CSS formats must be placed in the CSS file.

- ❖ **Hint:** Students must observe the web layout carefully before designing to organize the content into appropriate containers (div or span) and place them on the webpage. We must also specify their *id* or *class* property to access them later through the DOM model.

The hierarchical structure of the webpage is described as follows (see [Figure 17](#) for more details):

```
<body>  
  #banner  
  #title  
  #content  
    #rating_review  
      #rating  
        #review  
          .review_box  
            .reviewer  
  #general overview  
    #overview_text  
  #total
```


The properties used in this exercise are as follows (values to these properties are described in the requirement):

<body> : background-image, font-family, margin	(1)
#banner : width, background-image, text-align	(2)
#title : font-size, text-shadow, text-align	(3)
#content : width, border, border-radius, margin-left/right, overflow	(4)
#rating_review : width	(6)
#rating : background-image, height, color, font-size/weight	
 : vertical-align	
#review : padding-left/right, column-count/gap, margin-top	
.review_box : font-size/weight, border, border-radius, padding, overflow, background	
 : float, padding-right	
.reviewer : font-style, margin-bottom	(5)
 : float, margin-right	
:firstline : font-style	
#general overview : background-color, width, font-family/size, float	
#overview_text : margin	
<dt> : font-weight, margin-top	(5)
 : list-style-type, padding-left	
#total : background-color, padding, text-align, width, clear	(7)

The results after each step are given in the following links:

- 0) <http://bit.ly/1pO9hbh> (initial/unformatted page)
- 1) <http://bit.ly/1fQW1hK>
- 2) <http://bit.ly/1bV2gl3>
- 3) <http://bit.ly/1htRVwA>
- 4) <http://bit.ly/1lsnPwH>
- 5) <http://bit.ly/1hRzPmv>
- 6) <http://bit.ly/1fSAbu8>
- 7) Figure 16 (final result)

Ex 8. Create a simple calculator with CSSS and JS.

❖ **Objective:**

- 1) Event handling in JS.
- 2) Using the control structure and built-in function in JS.

❖ **Requirement:**

- 1) Design a simple calculator, as shown in [Figure 18](#).
- 2) The user can use the keyboard or click on the buttons of the calculator to input an expression which includes 2 operands and 1 operator (+, -, *, /)
- 3) Validate the user input. If the expression is valid, evaluate the expression and show the result. Otherwise, show an appropriate message.

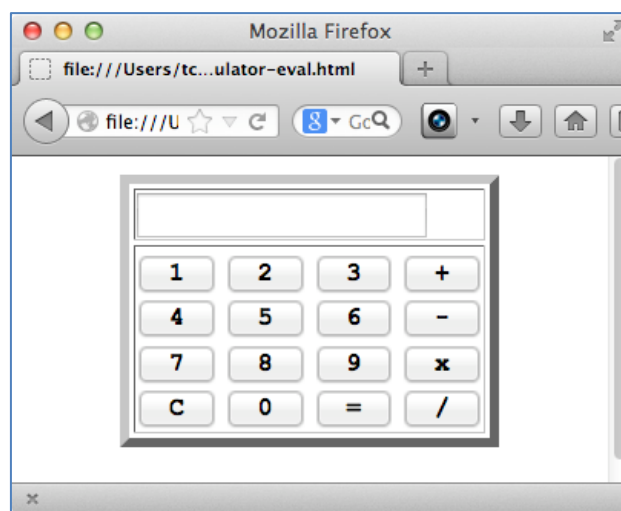


Figure 18. A simple calculator

❖ **Hint:**

- 1) UI design: 1 textfield and 16 buttons. User font Courier New for the text on the button.
- 2) Handle the mouse click event of the buttons:
 - Numbers and operators: append to button label to the textfield value
 - Equal (=) button: calculate the expression in the textfield (use the `eval()` function or write your own code to evaluate the expression) and display the result. To ensure the inputted expression is valid, we can use the regular expression to validate it.

Ex 9. Design a webpage that supports theme selection.

❖ **Objective:**

- 1) Use JS and DOM to change the property value of an element.
- 2) Use external CSS
- 3) Assign multiple classes to an element

❖ **Requirement:**

- 1) Modify the calculator in Ex 8: add a dropdown list as shown in Figure 19.1.
- 2) When the user chooses a theme in the dropdown list, the calculator interface will change accordingly, as shown in Figure 19.2 and Figure 19.3.

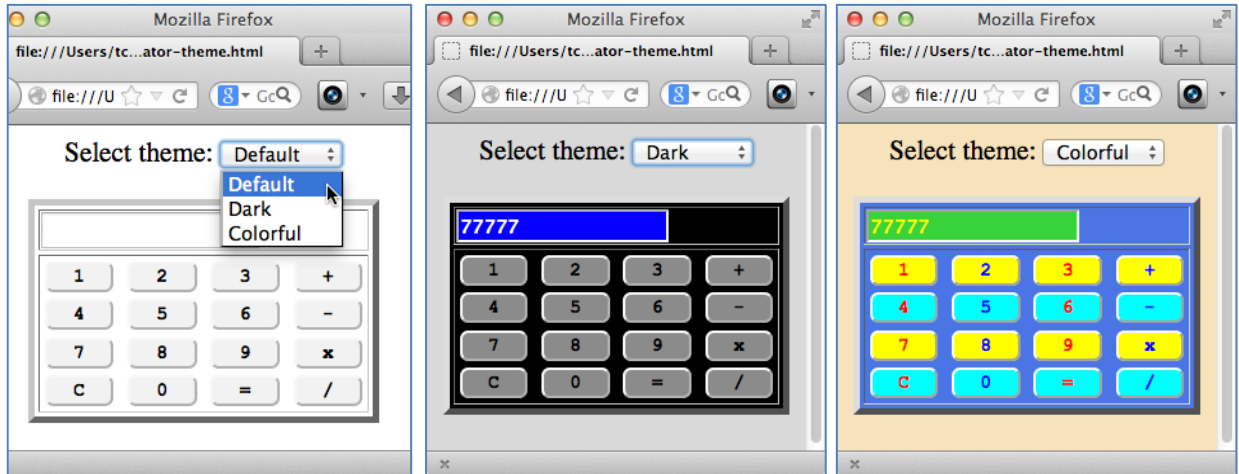


Figure 19. A calculator with themes

❖ **Hints:**

- 1) Create 3 CSS files for 3 different themes: default, dark and colorful theme.
- 2) By default, we use the default theme by importing the default CSS file into the webpage using the following tag:

```
<link href="default.css" type="text/css" rel="stylesheet" id="themeCSS"/>
```

We assign an *id* for this tag so that we can select it and change the CSS file later.
- 3) Write a JS function that gets a theme name and changes the *href* property of the `<link>` tag to the corresponding CSS file:

```
document.getElementById("themeCSS").href = <path to the CSS file>;
```
- 4) In the *onchange* event of the dropdown list (`<select>`), call the JS function to change the theme of the calculator based on the value selected in the dropdown list.